MISIÓN ÓRBITA

Primer avance: Preparación de la misión

Universidad CENFOTEC | SOFT-01 Principios de Programación 1 | Sección SCV6

Integrantes: [COMPLETAR]

Docente: Verónica Mora Lezcano

Fecha: [COMPLETAR]

Repositorio Git: [PEGAR ENLACE REAL]

Documento técnico basado en la consigna del curso. Las evidencias de consola incluidas son registros reales de pruebas automatizadas; antes de la entrega deben añadirse capturas realizadas en VS Code.

1. Alcance y restricciones

Se implementan cuatro algoritmos independientes. Ejercicio 1: secuencial; ejercicio 2: condicional simple; ejercicios 3 y 4: condicional doble. No se usan ciclos, colecciones, funciones propias, importaciones, elif ni decisiones anidadas. Las entradas numéricas se suponen enteros positivos conforme a la consigna.

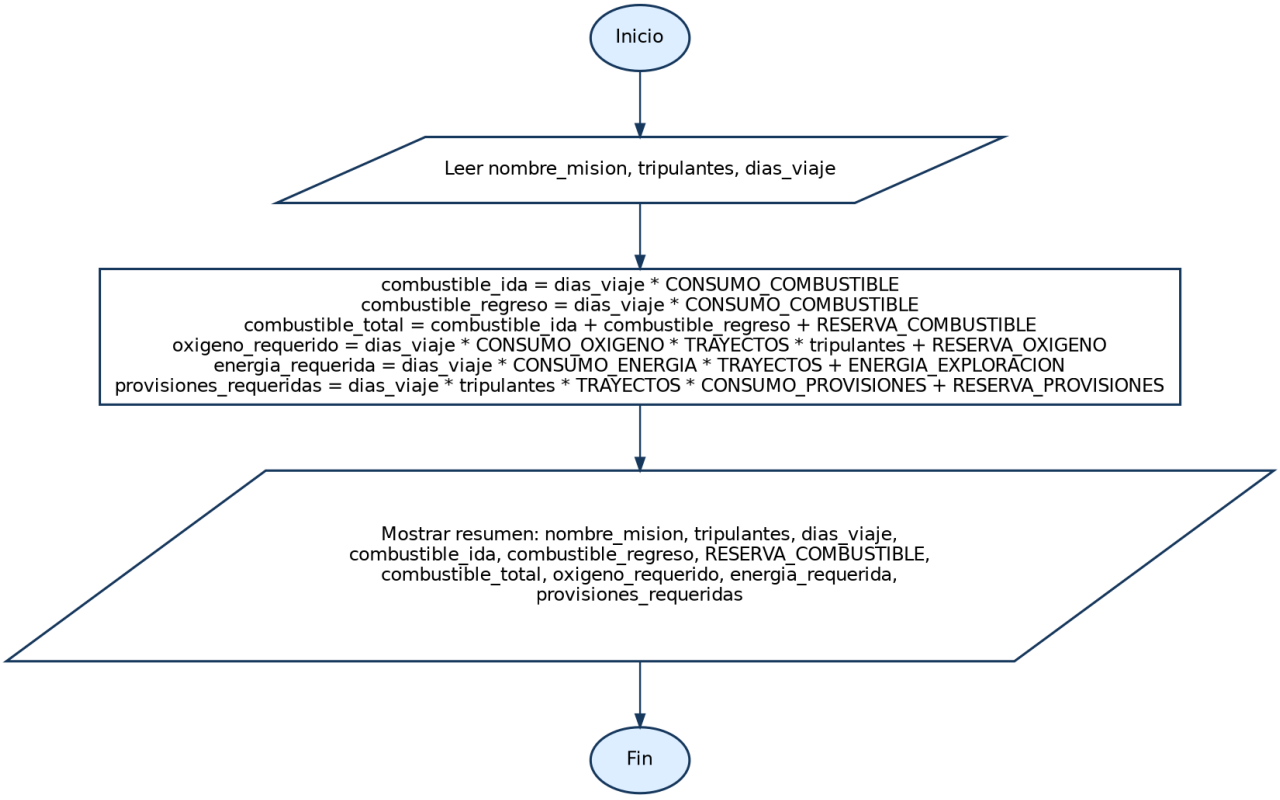
Orden metodológico: tabla de variables, diagrama de flujo, casos de prueba, implementación y verificación. Las constantes aparecen en las tablas con su valor inicial.

Ejercicio 1: Cálculo de recursos

Tabla de variables

Nombre	Tipo	Clasificación	Descripción / valor inicial
CONSUMO_COMBUSTIBLE	int	Constante	Unidades consumidas por dia | 8
RESERVA_COMBUSTIBLE	int	Constante	Reserva de emergencia | 10
CONSUMO_OXIGENO	int	Constante	Oxigeno diario por tripulante | 2
RESERVA_OXIGENO	int	Constante	Oxigeno adicional | 5
CONSUMO_ENERGIA	int	Constante	Energia diaria de nave | 5
ENERGIA_EXPLORACION	int	Constante	Energia para exploracion | 10
CONSUMO_PROVISIONES	int	Constante	Provision diaria por tripulante | 1
RESERVA_PROVISIONES	int	Constante	Provisiones extra | 3
TRAYECTOS	int	Constante	Ida y regreso | 2
nombre_mision	str	Entrada	Nombre identificador de mision | Usuario
tripulantes	int	Entrada	Numero de personas | Entero positivo
dias_viaje	int	Entrada	Duracion de ida en dias | Entero positivo
combustible_ida	int	Salida	Combustible del trayecto de ida | Calculado
combustible_regreso	int	Salida	Combustible del regreso | Calculado
combustible_total	int	Salida	Ida, regreso y reserva | Calculado
oxigeno_requerido	int	Salida	Oxigeno para ambos trayectos | Calculado
energia_requerida	int	Salida	Energia para trayectos y exploracion | Calculado
provisiones_requeridas	int	Salida	Alimento para ambos trayectos | Calculado

Ejercicio 1: Diagrama de flujo



Los identificadores y las condiciones corresponden al código. Cada diagrama tiene un único Inicio y Fin.

Ejercicio 1: Código Python

Archivo: ejercicio1_recursos.py

```
# ============================================================
# Archivo: ejercicio1_recursos.py
# Curso: SOFT-01 Principios de Programacion 1 - Seccion SCV6
# Integrantes: [COMPLETAR NOMBRES]
# Fecha: [COMPLETAR FECHA]    Version: 1.0
# Descripcion: Calcula los recursos para ida y regreso.
# ============================================================

# Declaracion de constantes
CONSUMO_COMBUSTIBLE = 8
RESERVA_COMBUSTIBLE = 10
CONSUMO_OXIGENO = 2
RESERVA_OXIGENO = 5
CONSUMO_ENERGIA = 5
ENERGIA_EXPLORACION = 10
CONSUMO_PROVISIONES = 1
RESERVA_PROVISIONES = 3
TRAYECTOS = 2

# Entrada de datos
nombre_mision = input("Nombre de la mision: ")
tripulantes = int(input("Cantidad de tripulantes: "))
dias_viaje = int(input("Dias estimados para llegar: "))

# Procesamiento: el viaje de regreso dura lo mismo que el de ida.
combustible_ida = dias_viaje * CONSUMO_COMBUSTIBLE
combustible_regreso = dias_viaje * CONSUMO_COMBUSTIBLE
combustible_total = combustible_ida + combustible_regreso + RESERVA_COMBUSTIBLE
oxigeno_requerido = dias_viaje * CONSUMO_OXIGENO * TRAYECTOS * tripulantes + RESERVA_OXIGENO
energia_requerida = dias_viaje * CONSUMO_ENERGIA * TRAYECTOS + ENERGIA_EXPLORACION
provisiones_requeridas = dias_viaje * tripulantes * TRAYECTOS * CONSUMO_PROVISIONES + RESERVA_PROVISIONES

# Salida: resumen completo con unidades.
print("\n--- RESUMEN DE LA MISION ---")
print("Mision:", nombre_mision)
print("Tripulantes:", tripulantes)
print("Duracion estimada hasta el destino:", dias_viaje, "dias")
print("Combustible para llegar:", combustible_ida, "unidades")
print("Combustible para regresar:", combustible_regreso, "unidades")
print("Reserva de combustible:", RESERVA_COMBUSTIBLE, "unidades")
print("Combustible total requerido:", combustible_total, "unidades")
print("Oxigeno requerido:", oxigeno_requerido, "unidades")
print("Energia requerida:", energia_requerida, "unidades")
print("Provisiones requeridas:", provisiones_requeridas, "unidades")
```

Ejercicio 1: Casos de prueba

ID / escenario	Datos de entrada (orden)	Resultado esperado	Resultado obtenido / estado
CP-01 | Ejemplo de la consigna	Explorador I, 3, 5	Combustible total requerido: 90 unidades ; Oxigeno requerido: 65 unidades ; Energia requerida: 60 unidades ; Provisiones requeridas: 33 unidades	Coincide con esperado / APROBADO
CP-02 | Mision corta, un tripulante	Orbita A, 1, 1	Combustible total requerido: 26 unidades ; Oxigeno requerido: 9 unidades ; Energia requerida: 20 unidades ; Provisiones requeridas: 5 unidades	Coincide con esperado / APROBADO
CP-03 | Mision larga, cuatro tripulantes	Orbita B, 4, 10	Combustible total requerido: 170 unidades ; Oxigeno requerido: 165 unidades ; Energia requerida: 110 unidades ; Provisiones requeridas: 83 unidades	Coincide con esperado / APROBADO

Verificación: los programas se ejecutaron con las entradas indicadas. A continuación se presentan fragmentos de la salida real de consola (registros de texto, no capturas de VS Code).

CP-01: Ejemplo de la consigna

```
Combustible total requerido: 90 unidades
Oxigeno requerido: 65 unidades
Energia requerida: 60 unidades
Provisiones requeridas: 33 unidades
```

CP-02: Mision corta, un tripulante

```
Combustible total requerido: 26 unidades
Oxigeno requerido: 9 unidades
Energia requerida: 20 unidades
Provisiones requeridas: 5 unidades
```

CP-03: Mision larga, cuatro tripulantes

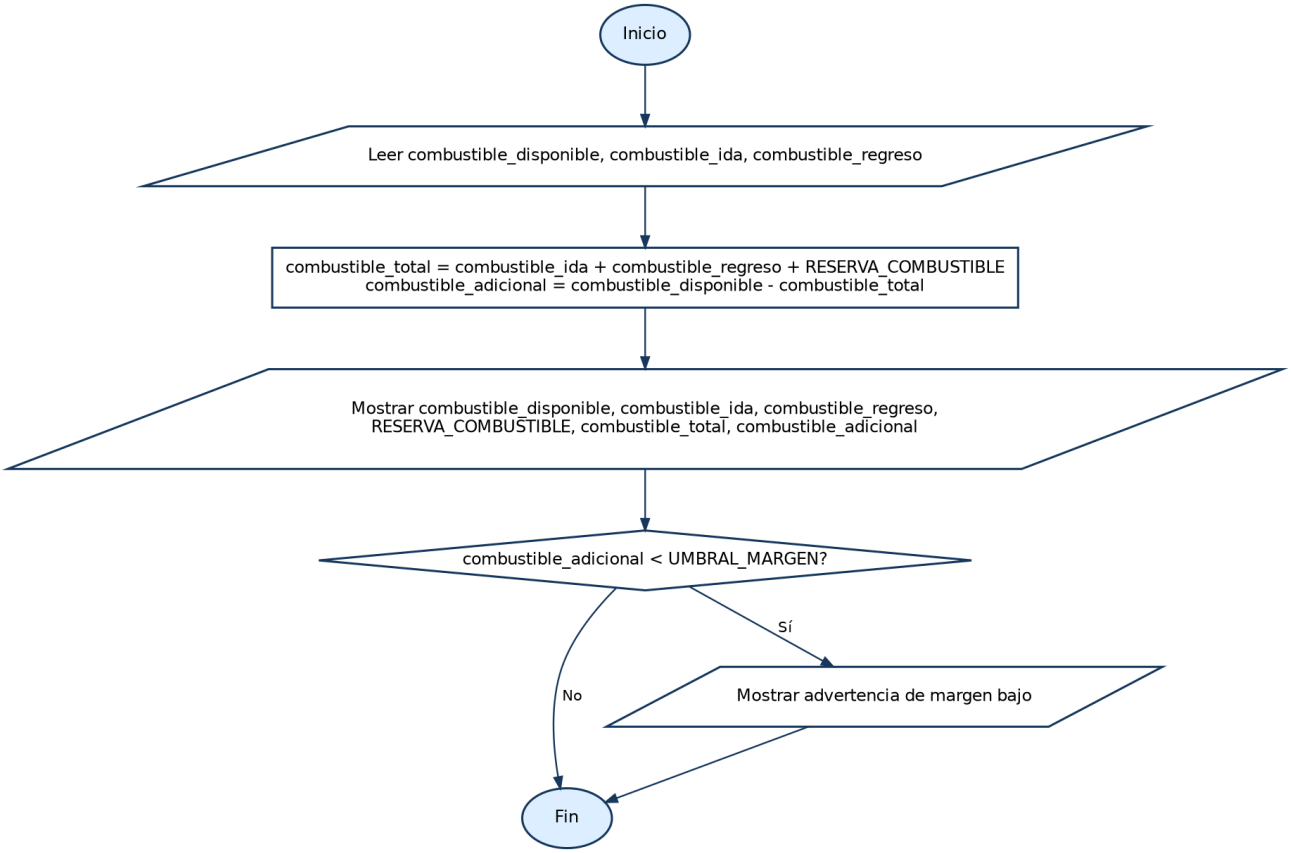
```
Combustible total requerido: 170 unidades
Oxigeno requerido: 165 unidades
Energia requerida: 110 unidades
Provisiones requeridas: 83 unidades
```

Ejercicio 2: Verificación de combustible

Tabla de variables

Nombre	Tipo	Clasificación	Descripción / valor inicial
RESERVA_COMBUSTIBLE	int	Constante	Reserva de emergencia | 10
UMBRAL_MARGEN	int	Constante	Margen minimo sin advertencia | 10
combustible_disponible	int	Entrada	Combustible actual | Entero positivo
combustible_ida	int	Entrada	Requerido para llegar | Entero positivo
combustible_regreso	int	Entrada	Requerido para volver | Entero positivo
combustible_total	int	Salida	Ida, regreso y reserva | Calculado
combustible_adicional	int	Salida	Disponible menos total; puede ser negativo | Calculado

Ejercicio 2: Diagrama de flujo



Los identificadores y las condiciones corresponden al código. Cada diagrama tiene un único Inicio y Fin.

Ejercicio 2: Código Python

Archivo: ejercicio2_combustible.py

```
# ============================================================
# Archivo: ejercicio2_combustible.py
# Curso: SOFT-01 Principios de Programacion 1 - Seccion SCV6
# Integrantes: [COMPLETAR NOMBRES]
# Fecha: [COMPLETAR FECHA]    Version: 1.0
# Descripcion: Verifica el margen adicional de combustible.
# ============================================================

# Declaracion de constantes
RESERVA_COMBUSTIBLE = 10
UMBRAL_MARGEN = 10

# Entrada de datos
combustible_disponible = int(input("Combustible disponible: "))
combustible_ida = int(input("Combustible requerido para llegar: "))
combustible_regreso = int(input("Combustible requerido para regresar: "))

# Procesamiento: el total incluye el regreso y la reserva.
combustible_total = combustible_ida + combustible_regreso + RESERVA_COMBUSTIBLE
combustible_adicional = combustible_disponible - combustible_total

# Salida obligatoria, incluso cuando el margen es insuficiente.
print("Combustible disponible:", combustible_disponible, "unidades")
print("Combustible para llegar:", combustible_ida, "unidades")
print("Combustible para regresar:", combustible_regreso, "unidades")
print("Reserva de emergencia:", RESERVA_COMBUSTIBLE, "unidades")
print("Combustible total requerido:", combustible_total, "unidades")
print("Combustible adicional:", combustible_adicional, "unidades")

# Decision simple: se advierte solo si el margen es menor que diez.
if combustible_adicional < UMBRAL_MARGEN:
    print("Advertencia: el margen adicional de combustible es bajo.")
```

Ejercicio 2: Casos de prueba

ID / escenario	Datos de entrada (orden)	Resultado esperado	Resultado obtenido / estado
CP-04 | Margen suficiente	110, 40, 40	Combustible total requerido: 90 unidades ; Combustible adicional: 20 unidades	Coincide con esperado / APROBADO
CP-05 | Margen exactamente 10	100, 40, 40	Combustible adicional: 10 unidades	Coincide con esperado / APROBADO
CP-06 | Margen bajo positivo	95, 40, 40	Combustible adicional: 5 unidades ; Advertencia: el margen adicional de combustible es bajo.	Coincide con esperado / APROBADO
CP-07 | Solo combustible de ida	40, 40, 40	Combustible adicional: -50 unidades ; Advertencia: el margen adicional de combustible es bajo.	Coincide con esperado / APROBADO

Verificación: los programas se ejecutaron con las entradas indicadas. A continuación se presentan fragmentos de la salida real de consola (registros de texto, no capturas de VS Code).

CP-04: Margen suficiente

Combustible total requerido: 90 unidades
Combustible adicional: 20 unidades

CP-05: Margen exactamente 10

Combustible total requerido: 90 unidades
Combustible adicional: 10 unidades

CP-06: Margen bajo positivo

Combustible total requerido: 90 unidades
Combustible adicional: 5 unidades
Advertencia: el margen adicional de combustible es bajo.

CP-07: Solo combustible de ida

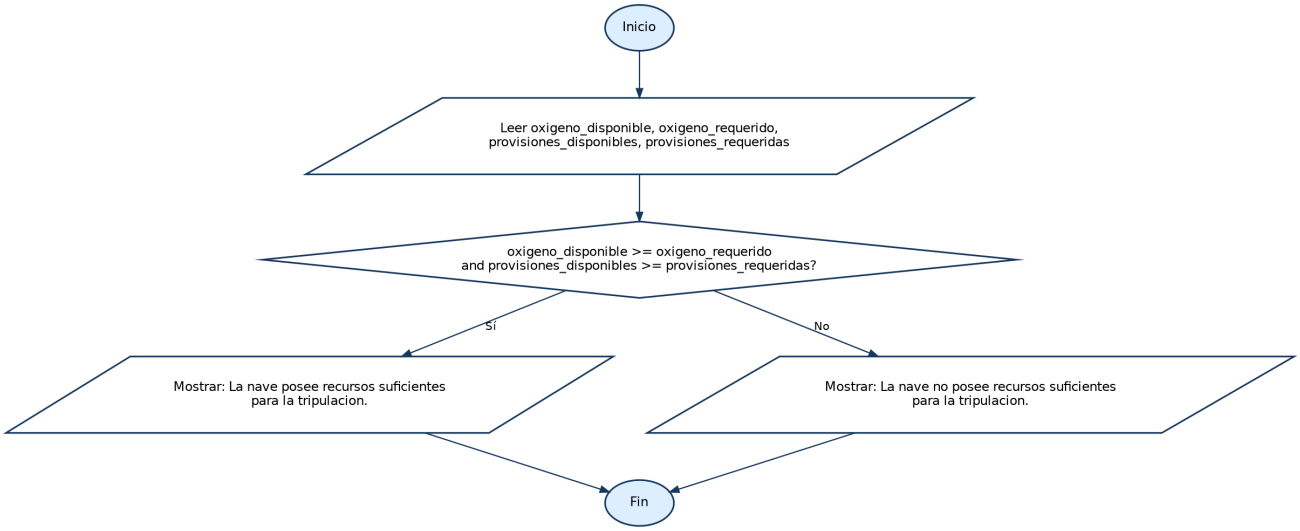
Combustible total requerido: 90 unidades
Combustible adicional: -50 unidades
Advertencia: el margen adicional de combustible es bajo.

Ejercicio 3: Soporte para la tripulación

Tabla de variables

Nombre	Tipo	Clasificación	Descripción / valor inicial
oxigeno_disponible	int	Entrada	Oxigeno disponible | Entero positivo
oxigeno_requerido	int	Entrada	Oxigeno minimo | Entero positivo
provisiones_disponibles	int	Entrada	Provisiones disponibles | Entero positivo
provisiones_requeridas	int	Entrada	Provisiones minimas | Entero positivo

Ejercicio 3: Diagrama de flujo



Los identificadores y las condiciones corresponden al código. Cada diagrama tiene un único Inicio y Fin.

Ejercicio 3: Código Python

Archivo: ejercicio3_soporte_tripulacion.py

```
# ============================================================
# Archivo: ejercicio3_soporte_tripulacion.py
# Curso: SOFT-01 Principios de Programacion 1 - Seccion SCV6
# Integrantes: [COMPLETAR NOMBRES]
# Fecha: [COMPLETAR FECHA]    Version: 1.0
# Descripcion: Verifica soporte vital de la tripulacion.
# ============================================================

# Entrada de datos
oxigeno_disponible = int(input("Oxigeno disponible: "))
oxigeno_requerido = int(input("Oxigeno requerido: "))
provisiones_disponibles = int(input("Provisiones disponibles: "))
provisiones_requeridas = int(input("Provisiones requeridas: "))

# Decision doble: ambos recursos deben cubrir el minimo requerido.
if oxigeno_disponible >= oxigeno_requerido and provisiones_disponibles >= provisiones_requeridas:
    print("La nave posee recursos suficientes para la tripulacion.")
else:
    print("La nave no posee recursos suficientes para la tripulacion.")
```

Ejercicio 3: Casos de prueba

ID / escenario	Datos de entrada (orden)	Resultado esperado	Resultado obtenido / estado
CP-08 | Ambos recursos suficientes	70, 65, 40, 33	La nave posee recursos suficientes para la tripulacion.	Coincide con esperado / APROBADO
CP-09 | Iguales al minimo	65, 65, 33, 33	La nave posee recursos suficientes para la tripulacion.	Coincide con esperado / APROBADO
CP-10 | Oxigeno insuficiente	64, 65, 40, 33	La nave no posee recursos suficientes para la tripulacion.	Coincide con esperado / APROBADO
CP-11 | Provisiones insuficientes	70, 65, 32, 33	La nave no posee recursos suficientes para la tripulacion.	Coincide con esperado / APROBADO

Verificación: los programas se ejecutaron con las entradas indicadas. A continuación se presentan fragmentos de la salida real de consola (registros de texto, no capturas de VS Code).

CP-08: Ambos recursos suficientes

Oxigeno disponible: Oxigeno requerido: Provisiones disponibles: Provisiones requeridas: La nave posee recursos suficientes para la tripulacion.

CP-09: Iguales al minimo

Oxigeno disponible: Oxigeno requerido: Provisiones disponibles: Provisiones requeridas: La nave posee recursos suficientes para la tripulacion.

CP-10: Oxigeno insuficiente

Oxigeno disponible: Oxigeno requerido: Provisiones disponibles: Provisiones requeridas: La nave no posee recursos suficientes para la tripulacion.

CP-11: Provisiones insuficientes

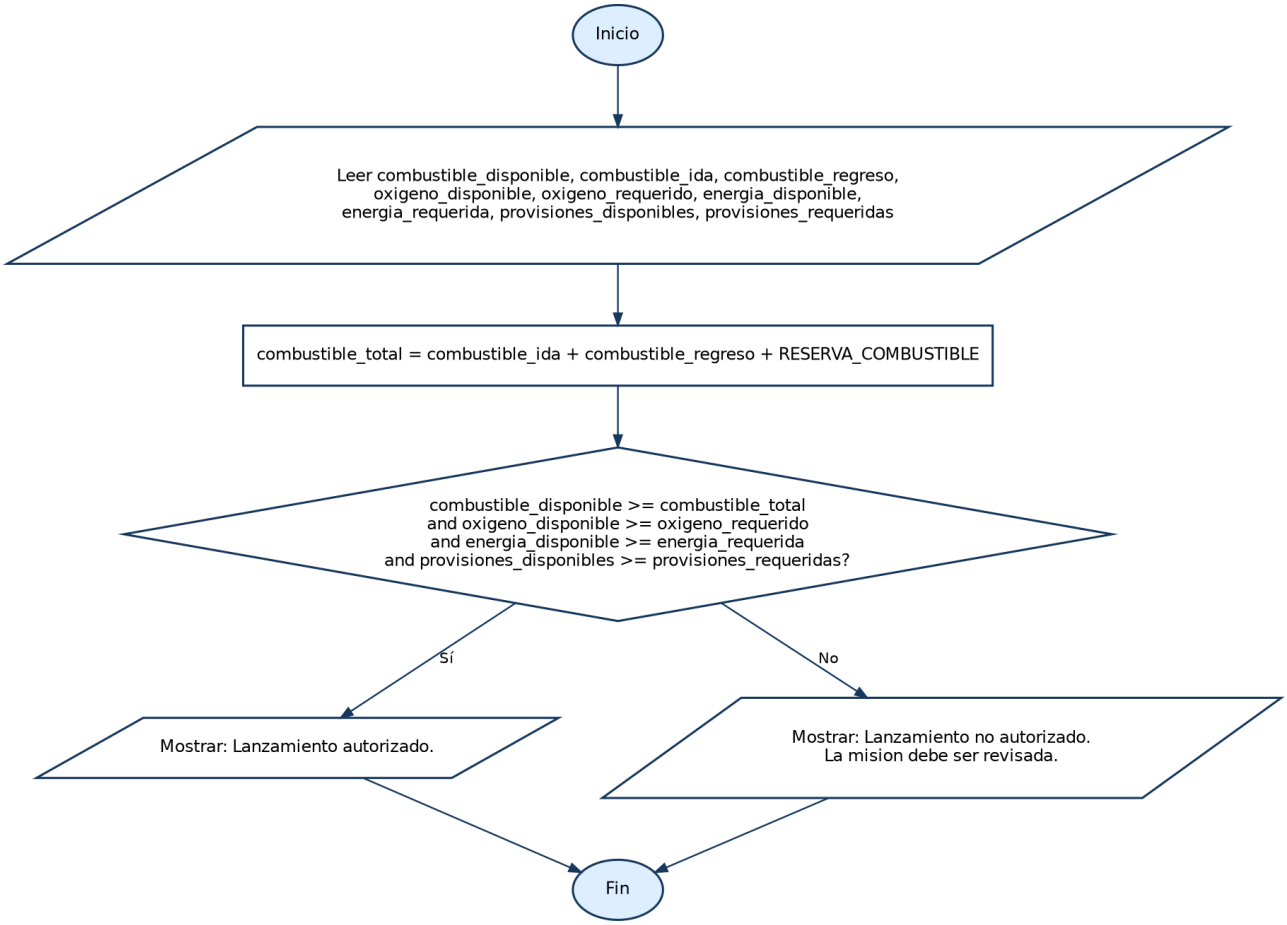
Oxigeno disponible: Oxigeno requerido: Provisiones disponibles: Provisiones requeridas: La nave no posee recursos suficientes para la tripulacion.

Ejercicio 4: Autorización de lanzamiento

Tabla de variables

Nombre	Tipo	Clasificación	Descripción / valor inicial
RESERVA_COMBUSTIBLE	int	Constante	Reserva obligatoria | 10
combustible_disponible	int	Entrada	Combustible actual | Entero positivo
combustible_ida	int	Entrada	Requerido para ida | Entero positivo
combustible_regreso	int	Entrada	Requerido para vuelta | Entero positivo
oxigeno_disponible	int	Entrada	Oxigeno disponible | Entero positivo
oxigeno_requerido	int	Entrada	Oxigeno requerido | Entero positivo
energia_disponible	int	Entrada	Energia disponible | Entero positivo
energia_requerida	int	Entrada	Energia requerida | Entero positivo
provisiones_disponibles	int	Entrada	Provisiones disponibles | Entero positivo
provisiones_requeridas	int	Entrada	Provisiones requeridas | Entero positivo
combustible_total	int	Intermedia	Combustible minimo con reserva | Calculado

Ejercicio 4: Diagrama de flujo



Los identificadores y las condiciones corresponden al código. Cada diagrama tiene un único Inicio y Fin.

Ejercicio 4: Código Python

Archivo: ejercicio4_autorizacion.py

```
# ============================================================
# Archivo: ejercicio4_autorizacion.py
# Curso: SOFT-01 Principios de Programacion 1 - Seccion SCV6
# Integrantes: [COMPLETAR NOMBRES]
# Fecha: [COMPLETAR FECHA]    Version: 1.0
# Descripcion: Autoriza lanzamiento si todos los recursos alcanzan.
# ============================================================

# Declaracion de constantes
RESERVA_COMBUSTIBLE = 10

# Entrada de datos
combustible_disponible = int(input("Combustible disponible: "))
combustible_ida = int(input("Combustible requerido para llegar: "))
combustible_regreso = int(input("Combustible requerido para regresar: "))
oxigeno_disponible = int(input("Oxigeno disponible: "))
oxigeno_requerido = int(input("Oxigeno requerido: "))
energia_disponible = int(input("Energia disponible: "))
energia_requerida = int(input("Energia requerida: "))
provisiones_disponibles = int(input("Provisiones disponibles: "))
provisiones_requeridas = int(input("Provisiones requeridas: "))

# Procesamiento: se reserva combustible tambien para el regreso.
combustible_total = combustible_ida + combustible_regreso + RESERVA_COMBUSTIBLE

# Decision doble: todas las condiciones deben cumplirse a la vez.
if (combustible_disponible >= combustible_total
        and oxigeno_disponible >= oxigeno_requerido
        and energia_disponible >= energia_requerida
        and provisiones_disponibles >= provisiones_requeridas):
    print("Lanzamiento autorizado.")
else:
    print("Lanzamiento no autorizado. La mision debe ser revisada.")
```

Ejercicio 4: Casos de prueba

ID / escenario	Datos de entrada (orden)	Resultado esperado	Resultado obtenido / estado
CP-12 | Recursos superiores	100, 40, 40, 70, 65, 65, 60, 40, 33	Lanzamiento autorizado.	Coincide con esperado / APROBADO
CP-13 | Todos exactamente al minimo	90, 40, 40, 65, 65, 60, 60, 33, 33	Lanzamiento autorizado.	Coincide con esperado / APROBADO
CP-14 | Solo combustible de ida	40, 40, 40, 70, 65, 65, 60, 40, 33	Lanzamiento no autorizado. La mision debe ser revisada.	Coincide con esperado / APROBADO
CP-15 | Combustible sin reserva	80, 40, 40, 70, 65, 65, 60, 40, 33	Lanzamiento no autorizado. La mision debe ser revisada.	Coincide con esperado / APROBADO
CP-16 | Energia insuficiente	100, 40, 40, 70, 65, 59, 60, 40, 33	Lanzamiento no autorizado. La mision debe ser revisada.	Coincide con esperado / APROBADO

Verificación: los programas se ejecutaron con las entradas indicadas. A continuación se presentan fragmentos de la salida real de consola (registros de texto, no capturas de VS Code).

CP-12: Recursos superiores

Combustible disponible: Combustible requerido para llegar: Combustible requerido para regresar: Oxigeno disponible: Oxigeno requerido: Energia disponible: Energia requerida: Provisiones disponibles: Provisiones requeridas: Lanzamiento autorizado.

CP-13: Todos exactamente al minimo

Combustible disponible: Combustible requerido para llegar: Combustible requerido para regresar: Oxigeno disponible: Oxigeno requerido: Energia disponible: Energia requerida: Provisiones disponibles: Provisiones requeridas: Lanzamiento autorizado.

CP-14: Solo combustible de ida

Combustible disponible: Combustible requerido para llegar: Combustible requerido para regresar: Oxigeno disponible: Oxigeno requerido: Energia disponible: Energia requerida: Provisiones disponibles: Provisiones requeridas: Lanzamiento no autorizado. La mision debe ser revisada.

CP-15: Combustible sin reserva

Combustible disponible: Combustible requerido para llegar: Combustible requerido para regresar: Oxigeno disponible: Oxigeno requerido: Energia disponible: Energia requerida: Provisiones disponibles: Provisiones requeridas: Lanzamiento no autorizado. La mision debe ser revisada.

CP-16: Energia insuficiente

Combustible disponible: Combustible requerido para llegar: Combustible requerido para regresar: Oxigeno disponible: Oxigeno requerido: Energia disponible: Energia requerida: Provisiones disponibles: Provisiones requeridas: Lanzamiento no autorizado. La mision debe ser revisada.

Repositorio y verificación final

Estructura: README.md, avance1/ (4 archivos .py), diagramas/ (4 PNG), documentacion/avance1.pdf y archivos de salida de pruebas.

El repositorio debe ser accesible para la docente, contener un README y registrar al menos ocho commits reales en días distintos. Todos los integrantes deben contribuir.

Pendiente: agregar nombres, fecha, enlace al repositorio y capturas reales de cada caso de prueba desde la terminal de VS Code.

Nota sobre la consigna: la primera página indica 44 puntos y 5%, pero la fórmula final menciona 64 puntos y 10%. Consultar con la docente; no se ha modificado ninguno de esos valores.